Java OCA – Chapter 6 - Übungen zur OOP

1 Gehaltserhöhung

Erstellen Sie eine Klasse für einen Mitarbeiter in einem Betrieb. Diese soll die Attribute Name, Vorname, Personalnummer und Monatslohn enthalten.

Schreiben Sie außerdem eine Methode, die es erlaubt den Monatslohn um einen bestimmten Prozentsatz zu erhöhen. Dieser soll sich variabel über das Hauptprogramm vorgeben lassen.

Erstellen Sie das Hauptprogramm, das ein entsprechendes Objekt erzeugt, ihm Werte zuweist und diese anschließend ausgibt. Rufen Sie daraufhin die Methode für die Gehaltserhöhung auf und geben Sie den neuen Lohn aus.

Hinweis: In dieser Übung soll kein parametrisierter Konstruktor verwendet werden!

2 Baum (Teil 1)

Baum
pflanzjahr: int ausdehnung: double hoehe: double form: String krank: boolean
alter( ): int setKrank(krank: boolean) isKrank():boolean setHoehe(hoehe: double) getHoehe():double setAusdehnung(ausdehnung:double) getAusdehnung( ): double showObjektVar( )

Schreiben Sie eine Klasse `Baum`. Folgende Objektvariablen beschreiben den Zustand eines Baumobjektes:

Objektvariablen

- `pflanzjahr` numerisch
  Ist eine vierstellige Zahl, die das Pflanzjahr des Baumes enthält.
- `ausdehnung` numerisch
  Gibt den Platzbedarf des ausgewachsenen Baumes in Quadratmetern an.
- `hoehe` numerisch
  Entspricht der aktuellen Höhe eines Baumes in Metern.
- `form` alphanumerisch
  Beschreibt die Wuchsform eines Baumes.
- `krank` boolean
  Zeigt an, ob ein Baum krank oder gesund ist.

Der Zustand eines Baumobjektes ist über die folgenden Objektmethoden änder- bzw. abrufbar.

Java OCA – Chapter 6 - Übungen zur OOP

Objektmethoden

- `alter( )`
  Bestimmt abhängig vom aktuellen Jahr und dem Pflanzjahr das Alter eines Baumes.
- `setKrank(boolean offen)`
  setzt die Objektvariable `krank` auf den übergebenen Wert.
- `getKrank( )`
  liefert den Wert der Objektvariablen `krank` zurück.
- `setHoehe(double hoehe)`
  setzt die Objektvariable `hoehe` auf den übergebenen Wert.
- `getHoehe()`
  liefert den Wert der Objektvariable `hoehe` zurück.
- `setAusdehnung(double ausdehnung)`
  setzt die Objektvariable `ausdehnung` auf den übergebenen Wert.
- `getAusdehnung( )`
  liefert den Wert der Objektvariable `ausdehnung` zurück.
- `showObjektVar( )`
  gibt alle Objektvariablen UND das Alter eines Baumes in der Konsole aus.

main()-Methode

Schreiben Sie eine zweite Klasse namens Start, die die `main()`-Methode und somit die Steuerung des Programms enthält. Die `main( )`-Methode soll dabei folgende Aufgaben erfüllen:

1 Erzeugen Sie zwei Baumobjekte.
2 Füllen Sie die Objektvariablen der Bäume mit realistischen, aber unterschiedlichen Werten.
3 Lassen Sie sich die Objektvariablen beider Bäume anzeigen.
4 Ändern Sie die Höhe eines Baumes.
5 Ändern Sie den Krankheitszustand des anderen Baumes.
6 Lassen Sie sich die Objektvariablen beider Bäume erneut anzeigen.

Hinweis

Das aktuelle Jahr kann in Java mit dem Befehl

```
Calendar cal = new GregorianCalendar( );
int jahr = cal.get(Calendar.YEAR);
```

bestimmt werden.

Anmerkung zur Datumsfunktion

Inzwischen nutzt man eine andere DATE-TIME-API - man nutzt heute statische Methoden - aber die „alte“ funktioniert nach wie vor.

Java OCA – Chapter 6 - Übungen zur OOP

3 Kuchen

Kuchen
mehl: int butter: int eier: int zucker: int teig: boolean stueckeGesamt: int stueckeVorhanden: int
backen( ) essen(anzahl: int) showObjektVar( )

Schreiben Sie eine Klasse Kuchen. Folgende Objektvariablen beschreiben den Zustand der Kuchenobjekte.

Objektvariablen

- `mehl, butter, zucker, eier` numerisch
  Mengenangaben für die Zutaten
- `teig` boolean
  wahr, wenn Teig / falsch, wenn Kuchen
- `stueckeGesamt` numerisch
- `stueckeVorhanden` numerisch
  Gesamtzahl und noch übrige Stücke des Kuchens.

Der Zustand eines Kuchenobjektes ist über die folgenden Objektmethoden änder- bzw. abrufbar.

Objektmethoden

- `backen( )`
  verändert den Zustand der Objektvariable teig.
- `essen(int anzahl)`
  Die Methode `essen( )` verringert den Wert der Variable `stueckeVorhanden` um die übergebene `anzahl`.
- `showObjectVar( )`
  gibt alle Objektvariablen in der Konsole aus.

Java OCA – Chapter 6 - Übungen zur OOP

main()-Methode

Schreiben Sie eine zweite Klasse namens Start, die die `main( )`–Methode und somit die Steuerung des Programms enthält. Die `main( )`–Methode soll dabei folgende Aufgaben erfüllen:

1. Erzeugen Sie zwei Kuchenobjekte.
2. Füllen Sie die Objektvariablen der Kuchen mit realistischen, aber unterschiedlichen Werten.
3. Lassen Sie sich die Objektvariablen beider Kuchen anzeigen.
4. Backen Sie beide Kuchen.
5. Essen Sie vom ersten Kuchen vier und vom zweiten Kuchen drei Stücke.
6. Lassen Sie sich die Objektvariablen beider Kuchen erneut anzeigen.

Zusatzaufgabe

Verhindern Sie, dass mehr Stücke gegessen werden als noch vorhanden sind. Die Zutatenvariablen `mehl, zucker, butter, eier` und `zucker` geben nicht wirklich den Zustand des Kuchenobjektes wieder. Sonst dürfte nachdem der halbe Kuchen aufgegessen worden ist, auch nur noch die Hälfte der Zutatenmengen im Restkuchen vorhanden sein. Ändern Sie die Methode `essen` so ab, dass sich die Zutatenvariablen dynamisch dem verbleibenden Restkuchen anpassen. Kontrollieren Sie die Wahl der Datentypen!

Java OCA – Chapter 6 - Übungen zur OOP

4 Baum (Teil 2)

Baum
-pflanzjahr: int -ausdehnung: double -hoehe: double -form: String -krank: boolean
+<<construct>>Baum( ) +<<construct>>Baum(pflanzjahr: int) +<<construct>>Baum(pflanzjahr: int, ausdehnung: double, hoehe: double, form: String)
+alter( ): int +setKrank(krank: boolean) +isKrank():boolean +setHoehe(hoehe: double) +getHoehe():double +setAusdehnung(ausdehnung:double) +getAusdehnung( ): double +showObjektVar( )

Führen Sie an der Klasse Baum folgende Änderungen und Ergänzungen durch:

Objektvariablen
Kapseln Sie die Objektvariablen!

Konstruktoren
Fügen Sie folgende Konstruktoren hinzu:

- `Baum( )` Standardkonstruktor
  Ein Konstruktor, der einen Baum mit folgenden Standardwerten erzeugt:

```
pflanzjahr = 2000;          ausdehnung = 5.0d;
hoehe = 1.0d;               form = "rundlich";
krank = false;
```

- `Baum(int pflanzjahr)`
  Der Konstruktor erzeugt einen Standardbaum, allerdings mit einem individuellen Pflanzjahr.
- `Baum (int pflanzjahr, double ausdehnung, doube hoehe, String form)`
  Nur Bäume, die eine Mindesthöhe von einem Meter erreicht haben und gesund sind, werden im Botanischen Garten ausgepflanzt. Daher sind neu erzeugte Baumobjekte auf jeden Fall zunächst gesund. Alle anderen Objektvariablen müssen über diesen Konstruktor individuell eingestellt werden könne.

Java OCA – Chapter 6 - Übungen zur OOP

Wichtig
Verketten Sie die Konstruktoren miteinander.

Objektmethoden
Lokale Variablen sollten denselben Namen tragen wie ihre zugehörigen Objektvariablen. Um mögliche Namenskollisionen zu vermeiden, ist es sinnvoll Objektvariablen konsequent über ihre `this`-Referenz anzusprechen.

- `setHoehe(double hoehe)`
  Die Methode soll nur noch Höhen von mehr als einem Meter zulassen. Ist die übergebene Höhenangabe kleiner, ist die Objektvariable hoehe automatisch auf den Wert 1.0d einzustellen.

main()-Funktion
Ändern Sie die `main()`–Funktion so ab, dass nun den Objektvariablen der Baumobjekte die Werte über die Konstruktoren zugewiesen werden.

1. Erzeugen Sie drei verschiedene Bäume `b1` bis `b3`:
   - einen Standardbaum `b1`.
   - einen 2004 gepflanzten Standardbaum.
   - einen 1999 gepflanzten, 6.5 m hohen, säulenförmigen Baum mit einer Ausdehnung von 15.5 m^2.
2. Lassen Sie sich die Objektvariablen aller Bäume ausgeben.
3. Ändern Sie die Höhe des ersten Baumes `b1` auf 4.5 m.
4. Die Bäume `b2` und `b3` erkranken.
5. Lassen Sie sich die Objektvariablen aller Bäume erneut ausgeben.

Java OCA – Chapter 6 - Übungen zur OOP

5 Health-App (Schrittzähler)

Mobile Schrittzähler-Apps sind spätestens seit dem Erscheinen von Smartwatches durchaus populär. Das Grundgerüst für eine solche App sollen Sie in dieser Aufgabe programmieren. Das Programm soll jeden Schritt zählen und anschließend die gezählten Schritte mit dem Datum ausgeben. Zwar können wir am Computer die gegangen Schritte aktuell nur simulieren, aber praktisch lässt sich das Grundgerüst auch auf eine mobile App mit echter Schrittzählung übertragen.

Aufgabenstellung

Schreiben Sie eine Klasse mit dem Namen `Schrittzaehler`, dessen Konstruktor ein Datum in Form eines Strings entgegennehmen soll. Schreiben Sie in der Klasse die Methode `erhoeheAnzahlSchritte( )`, die den Schrittzähler um einen Schritt erhöhen soll.

Die Methode `toString( )` soll anschließend eine Ausgabe in dieser Form erzeugen:

```
Am <Datum> bin ich <Schrittzahl> Schritte gegangen.
```

Die umklammerten Worte sollen Platzhalter für die entsprechenden Werte sein.

Schreiben Sie anschließend ein Testprogramm außerhalb der Klasse, welches den Schrittzaehler teste und eine bestimmte Anzahl an Schritten simuliert.

Testfälle

- ```
  Datum: 11.12.2022
  Anzahl der Schritte: 1111
  Ausgabe:
  Am 11.12.2022 bin ich 1111 Schritte gegangen.
  ```
- ```
  Datum: 13.03.2023
  Anzahl der Schritte: 10000
  Ausgabe:
  Am 13.03.2023 bin ich 10000 Schritte gegangen.
  ```

Algorithmische Tipps

Wenn Sie stocken und nicht mehr weiterwissen, dann versuchen Sie mal Folgendes:

- Private Variablen sind hier empfehlenswert. Sonst könnten unehrliche Fußgänger hier einfach ihre Schritte ändern ;-).
- Mit jedem Aufruf der Schritterhöhung den Zähler um eins erhöhen.
- Eventuell muss der Variablentyp umgewandelt werden.

Java OCA – Chapter 6 - Übungen zur OOP

6 Parteistimmen

Bei Wahlen ist es immer interessant zu sehen, wie viel Prozent bestimme Parteien im Vergleich zur vorherigen Wahl gewonnen oder verloren haben. Zur Berechnung wollen wir dafür eine eigene Klasse schreiben, welche folgende Funktionen haben soll:

- Der Konstruktor speichert den Namen der Partei, die Anzahl der Stimmen in der vorherigen Wahl und die Anzahl der Stimmen in der aktuellen Wahl.
- Die Funktion `bestimmeEntwicklung ( )` berechnet, wie viel Prozent die Partei im Vergleich zur vorherigen Wahl gewonnen bzw. verloren hat. Berechnen können Sie die Entwicklung mit folgender Formel:

$$Entwicklung = \frac{StimmenAktuell - StimmenVorher}{StimmenVorher} \cdot 100$$

- Die Funktion `istZuwachs( )` gibt als Boolean-Wert `true` zurück, wenn die Partei im Vergleich zur vorherigen Wahl an Stimmen hinzugewonnen hat.
- Die `toString`-Funktion soll den Namen der Partei ausgeben, ob diese an Stimmen gewonnen oder verloren hat sowie den Gewinn/Verlust in Prozent.

Aufgabenstellung

Schreiben Sie die Klasse `Partei`, welche die Entwicklung der Parteistimmen mit den oben genannten Eigenschaften berechnet.

Testfälle

- Parteiname: Gewinnerpartei
  - 500 Stimmen in vorheriger Wahl
  - 600 Stimmen in aktueller Wahl
  - Ausgabe:
    `Die Partei 'Gewinnerpartei' hat einen Zuwachs von 20.0 % bekommen.`
- Parteiname: Verliererpartei
  - 600 Stimmen in vorheriger Wahl
  - 300 Stimmen in aktueller Wahl
  - Ausgabe:
    `Die Partei 'Verliererpartei' hat einen Verlust von 50.0 % bekommen.`

Algorithmische Tipps

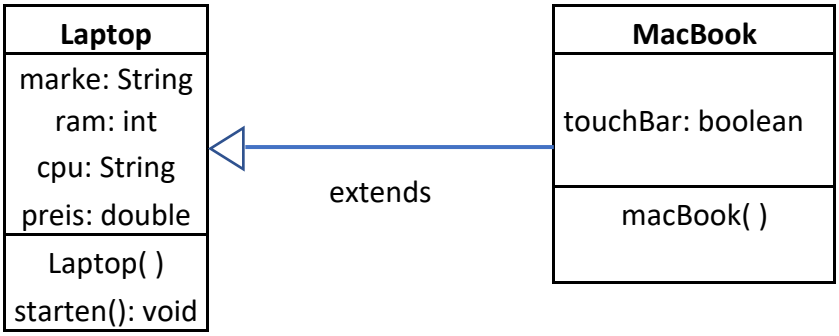
Wenn Sie stocken und nicht mehr weiterwissen, dann versuchen Sie mal Folgendes:

- Bei der Division könnten die Nachkommastellen abgeschnitten werden, sollten Sie den Integer-Datentypen dort verwenden.
- Vielleicht hilft es Ihnen, wenn Sie Integer-Zahlen in einen Datentyp mit Kommas umwandeln.
- Beim Zuwachs der Parteistimmen zur vorherigen Wahl ist die Prozentzahl positiv und andernfalls negativ.

Java OCA – Chapter 6 - Übungen zur OOP

7 Laptop

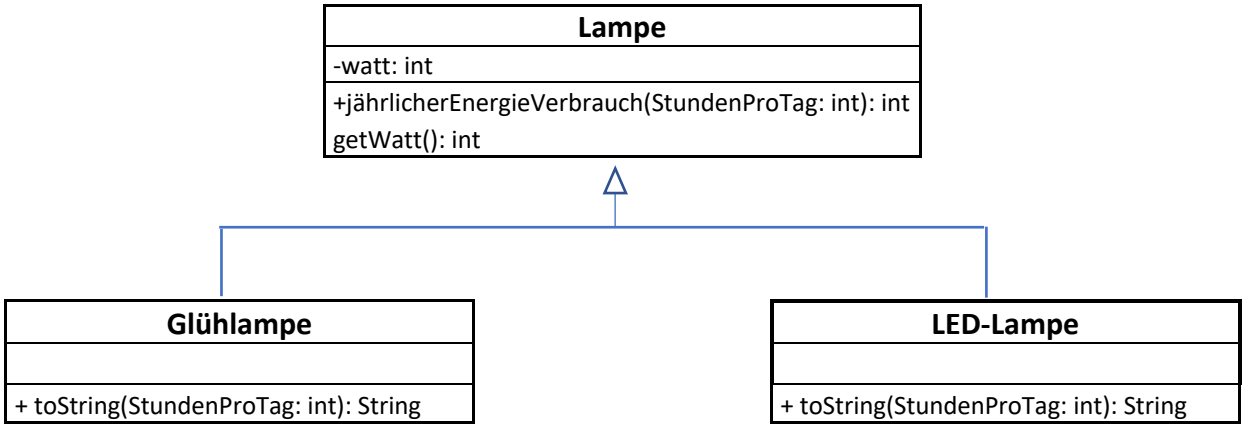
Zunächst das UML-Diagramm



1. Erstellen Sie die Klasse `Laptop` und `MacBook`
2. Erstellen Sie in der Klasse `Laptop` einen Konstruktor der die Parameter übernehmen kann.
3. Erstellen Sie in der Klasse `MacBook` einen Konstruktor der mit Hilfe des Schlüsselwortes `super( )` den Konstruktor von Laptop aufruft.
4. Überschreiben Sie die Methode `starten( )` aus der Klasse Laptop. Die Methode soll die Nachricht „MacBook startet“ ausgeben.
5. Erstellen Sie ein Objekt (Name: macBook) vom Typ MacBook (Werte: Apple, 16 GB, Intel i7, 1500 €, touchBar = ja).
6. Rufen Sie abschließend die Methode `starten( )` über das Objekt auf.

8 Lampen

Zur Repräsentierung von Lampen haben wir folgendes UML-Klassendiagramm bekommen:



Mit der Methode `jaehrlicherEnergieVerbrauch( )` können wir in Abhängigkeit des Parameters `StundenProTag`, der die Anzahl an Betriebsstunden pro Tag angibt, den jährlichen Energieverbrauch in Kilowattstunden (kWh) wie folgt berechnen:

```
(Leistung [Watt] * Betriebsstunden am Tag * 365) / 1000
```

Aufgabenstellung

Implementieren Sie das dargestellte UML-Klassendiagramm.
Schreiben Sie anschließend ein Programm, das den jährlichen Energieverbrauch einer 9-W-LED-Glühlampe und einer 60-W-Glühlampe berechnet und in der Konsole ausgibt. Als tägliche Betriebszeit werden 10 Stunden vorausgesetzt.

Testfälle

Korrekt implementiert sollte beim Aufruf der `toString( )`-Methoden eine unterschiedliche Ausgabe in der Konsole erfolgen. Hier eine Beispielausgabe:

```
Eine Glühlampe verbraucht 219 kWh pro Jahr.
Eine LED-Lampe verbraucht 32 kWh pro Jahr.
```

Java OCA – Chapter 6 - Übungen zur OOP

9 Gibt es einen Standardkonstruktor?

……

……

……

10 Ist doch „super“! Oder nicht?

Wie kann man aus einer überschriebenen Methode heraus die gleichnamige Methode der Basisklasse aufrufen?

……

……

……

……

……

……

11 Methoden

Wie kann eine Methode mit ihrer Umwelt Daten austauschen?

……

……

……

……

……

……

12 Sichtbarkeit

Wie kann man die Sichtbarkeit von Variablen steuern?

……

……

……

……

……

……

Java OCA – Chapter 6 - Übungen zur OOP

13 Schrank

Implementieren Sie eine Klasse Schrank mit den Instanzvariablen `name` für den Modellnamen, einem Array `abmessung` für die Länge, Breite und Höhe und `preis` für den Preis. Erstellen Sie drei Konstruktoren für zwei Einheitsschränke sowie für einen allgemeinen Schranktyp. Schreiben Sie neben den notwendigen Getter- und Setter Methoden noch die folgenden Methoden:

- `double getVolumen( )`
  Berechnen des Volumens eines Schrankes.
- `double getPreis( )`
  Berechnen des Preises eines Schrankes über eine geeignete Vorschrift aus dem Volumen.
- `String getInfo( )`
  Ausgabe der vollständigen Schrankeigenschaften als Zeichenkette.
- `int getAnzahl( )`
  Rückgabe der Anzahl der definierten Schränke mit Hilfe der Klassenvariablen `zaehler`.

Testen Sie anschließend diese Klasse mit mehreren definierten Schrankobjekten.